

PostgreSQL Common Queries — b-secure Reference

Production query reference for the b-secure platform. Covers Django ORM patterns and raw SQL equivalents for every major domain.

Table of Contents

- [1. ORM vs Raw SQL](#)
- [2. Guard Proximity Search](#)
- [3. Booking Lifecycle Queries](#)
- [4. Payment / Wallet Queries](#)
- [5. Analytics / Aggregation Queries](#)
- [6. Review & Rating Queries](#)
- [7. Admin Dashboard Queries](#)
- [8. Tracking / Location Queries](#)
- [9. Search Queries](#)
- [10. Performance Tips](#)

1. ORM vs Raw SQL

Criteria	Use Django ORM	Use Raw SQL
CRUD operations	✓ Always	✗
Simple filters / relations	✓ Always	✗
PostGIS spatial functions	✓ <code>django.contrib.gis.annotate</code>	✓ When ORM lacks the function
Complex aggregations	✓ <code>annotate + values</code>	✓ When window functions needed
Bulk inserts (>1000 rows)	✓ <code>bulk_create</code>	✓ <code>COPY FROM STDIN</code>
Recursive CTEs / LATERAL	✗ Not supported in ORM	✓ <code>connection.execute</code>

Criteria	Use Django ORM	Use Raw SQL
KNN nearest-neighbour (<->)	✗ No ORM support	✓ Only raw SQL
Analytics / reporting queries	⚠ Use if readable	✓ Preferred for clarity
Database migrations	✓ Always (schema changes)	✗

Golden rule: Use the ORM by default. Drop to raw SQL only when the ORM cannot express the query cleanly or when a critical index (like GiST/KNN) is bypassed.

Raw SQL execution pattern – always use parameterised queries
from django.db **import** connection

```
def raw_query(lat: float, lon: float, radius_m: int):
    with connection.cursor() as cursor:
        cursor.execute("""
            SELECT id, full_name, ST_Distance(location::geography,
            ST_MakePoint(%s, %s)::geography) AS distance
            FROM guards_guard
            WHERE ST_DWithin(location::geography,
            ST_MakePoint(%s, %s)::geography, %s)
            ORDER BY distance
            LIMIT 20
            """, [lon, lat, lon, lat, radius_m])
        columns = [col[0] for col in cursor.description]
        return [dict(zip(columns, row)) for row in
            cursor.fetchall()]
```

2. Guard Proximity Search

The most performance-critical query in the platform. Guards are stored with a PostGIS geography column named `location`.

2.1 Django ORM – Annotate + Filter

```
from django.contrib.gis.db.models.functions import Distance
from django.contrib.gis.geos import Point
from django.contrib.gis.measure import D
from guards.models import Guard

def get_nearby_guards_orm(lat: float, lon: float, radius_km:
    float = 5.0):
    """
    Returns active, verified guards within `radius_km` kilometres,
    ordered by distance ascending.
```

```

"""
user_location = Point(lon, lat, srid=4326) # SRID 4326 =
WGS84

return (
    Guard.objects
        .filter(
            is_active=True,
            is_verified=True,
            is_available=True,
            location__dwithin=(user_location, D(km=radius_km)),
        )
        .annotate(distance=Distance("location", user_location))
        .select_related("tier", "user")
        .order_by("distance")
)

```

Geography type note: The location column must be declared as geography(Point, 4326) in PostGIS (not geometry). Geography uses metres for distance calculations and accounts for Earth's curvature. Always cast in raw SQL as ST_MakePoint(lon, lat)::geography. The Django D(km=...) filter handles the conversion automatically when the field is a geography type.

2.2 Raw SQL — ST_DWithin + ST_Distance

```

-- Find all available guards within 5 km of (lat=12.9716,
lon=77.5946)
-- PostGIS geography uses metres, so 5 km = 5000 m
SELECT
    g.id,
    g.full_name,
    g.hourly_rate,
    g.tier_id,
    g.average_rating,
    ST_Distance(
        g.location::geography,
        ST_MakePoint(77.5946, 12.9716)::geography
    ) AS distance_metres
FROM guards_guard g
WHERE
    g.is_active      = TRUE
    AND g.is_verified = TRUE
    AND g.is_available = TRUE
    AND ST_DWithin(
        g.location::geography,
        ST_MakePoint(77.5946, 12.9716)::geography,
        5000          -- radius in metres
    )
ORDER BY distance_metres ASC
LIMIT 20;

```

Required index:

```
CREATE INDEX idx_guard_location_gist
  ON guards_guard USING GIST (location);
-- For geography column:
CREATE INDEX idx_guard_location_geography_gist
  ON guards_guard USING GIST (location::geography);
```

2.3 KNN Query — Nearest N Guards (No Radius)

Use when you want exactly N nearest guards regardless of distance (e.g., “show me the 5 nearest guards”).

```
-- KNN using the <=> distance operator – uses GiST index
  efficiently
-- Note: <=> works on geometry; cast back for accurate display
  distance

SELECT
  g.id,
  g.full_name,
  g.average_rating,
  g.location <=> ST_MakePoint(77.5946, 12.9716)::geometry AS
    knn_distance
FROM guards_guard g
WHERE
  g.is_active      = TRUE
  AND g.is_available = TRUE
ORDER BY g.location <=> ST_MakePoint(77.5946, 12.9716)::geometry
LIMIT 5;
```

KNN with <=> uses the GiST index in an index scan mode and is significantly faster than a full ST_DWithin scan when no radius is needed. The returned distance is in degrees (geometry SRID 4326), not metres — use ST_Distance(...::geography, ...::geography) in a second pass or CTE if you need metres.

2.4 Full NearbyGuardsView (DRF)

```
# guards/views.py
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework.permissions import IsAuthenticated
from django.contrib.gis.db.models.functions import Distance
from django.contrib.gis.geos import Point
from django.contrib.gis.measure import D
from guards.models import Guard
from guards.serializers import NearbyGuardSerializer

class NearbyGuardsView(APIView):
    permission_classes = [IsAuthenticated]
```

```

def get(self, request):
    try:
        lat = float(request.query_params["lat"])
        lon = float(request.query_params["lon"])
        radius_km =
float(request.query_params.get("radius_km", 5.0))
        limit = min(int(request.query_params.get("limit",
20)), 50)
    except (KeyError, ValueError):
        return Response({"detail": "lat and lon are
required."}, status=400)

    user_location = Point(lon, lat, srid=4326)

    guards = (
        Guard.objects
        .filter(
            is_active=True,
            is_verified=True,
            is_available=True,
            location__dwithin=(user_location,
D(km=radius_km)),
        )
        .annotate(distance=Distance("location",
user_location))
        .select_related("tier", "user")
        .only(
            "id", "full_name", "average_rating",
"hourly_rate",
            "profile_photo", "tier__name", "tier__color",
            "user__id", "location",
        )
        .order_by("distance")[:limit]
    )

    serializer = NearbyGuardSerializer(
        guards, many=True, context={"request": request}
    )
    return Response({"results": serializer.data, "count":
len(serializer.data)})

```

```

# guards/serializers.py
from rest_framework import serializers
from guards.models import Guard

```

```

class NearbyGuardSerializer(serializers.ModelSerializer):
    distance_km = serializers.SerializerMethodField()
    tier_name = serializers.CharField(source="tier.name",
read_only=True)
    tier_color = serializers.CharField(source="tier.color",
read_only=True)

```

```

class Meta:
    model = Guard
    fields = [
        "id", "full_name", "average_rating", "hourly_rate",
        "profile_photo", "tier_name", "tier_color",
        "distance_km",
    ]

def get_distance_km(self, obj) -> float:
    if hasattr(obj, "distance") and obj.distance:
        return round(obj.distance.km, 2)
    return None

```

3. Booking Lifecycle Queries

3.1 Create Booking (Atomic Transaction)

```

from django.db import transaction
from bookings.models import Booking
from payments.models import WalletTransaction
from decimal import Decimal

@transaction.atomic
def create_booking(user, guard, scheduled_at, hours: float,
                  address: str):
    """
    Creates a booking and pre-authorises the wallet balance in a
    single
    atomic transaction. Rolls back entirely if wallet is
    insufficient.
    """
    hourly_rate = guard.hourly_rate
    estimated_amount = Decimal(str(hourly_rate)) *
        Decimal(str(hours))

    # Lock user wallet row to prevent concurrent debits
    wallet =
        user.wallet.__class__.objects.select_for_update().get(user=user)

    if wallet.balance < estimated_amount:
        raise ValueError("Insufficient wallet balance.")

    booking = Booking.objects.create(
        user=user,
        guard=guard,
        scheduled_at=scheduled_at,
        estimated_hours=hours,
        address=address,
        estimated_amount=estimated_amount,

```

```

        status="pending",
    )

    # Hold the amount (deduct from available balance)
    wallet.balance -= estimated_amount
    wallet.on_hold += estimated_amount
    wallet.save(update_fields=["balance", "on_hold"])

    WalletTransaction.objects.create(
        wallet=wallet,
        booking=booking,
        amount=-estimated_amount,
        transaction_type="hold",
        description=f"Hold for booking #{booking.id}",
    )

    return booking

```

3.2 Accept Booking — Prevent Double-Acceptance

```

from django.db import transaction
from bookings.models import Booking

@transaction.atomic
def accept_booking(guard, booking_id: int):
    """
    Guard accepts a pending booking. Uses SELECT FOR UPDATE so
    that two
    guards (or two concurrent requests) cannot both accept the
    same booking.
    """
    try:
        booking = (
            Booking.objects
            .select_for_update(nowait=True)  # raises
            OperationalError if locked
            .get(id=booking_id, status="pending")
        )
    except Booking.DoesNotExist:
        raise ValueError("Booking not found or already accepted.")

    if booking.guard_id != guard.id:
        raise PermissionError("This booking is not assigned to
        you.")

    booking.status = "accepted"
    booking.accepted_at = timezone.now()
    booking.save(update_fields=["status", "accepted_at"])
    return booking

```

3.3 Complete Booking — Duration + Final Price

```
from django.utils import timezone
from decimal import Decimal

@transaction.atomic
def complete_booking(booking_id: int, guard):
    booking = (
        Booking.objects
        .select_for_update()
        .select_related("user__wallet")
        .get(id=booking_id, guard=guard, status="active")
    )

    now = timezone.now()
    duration_hrs = (now - booking.started_at).total_seconds() /
        3600
    final_amount = Decimal(str(guard.hourly_rate)) *
        Decimal(str(round(duration_hrs, 2)))

    booking.status = "completed"
    booking.completed_at = now
    booking.actual_hours = round(duration_hrs, 2)
    booking.final_amount = final_amount
    booking.save(update_fields=["status", "completed_at",
        "actual_hours", "final_amount"])

    wallet = booking.user.wallet
    held = booking.estimated_amount
    refund = max(held - final_amount, Decimal("0"))

    # Release hold, charge actual amount
    wallet.on_hold -= held
    wallet.balance += refund # refund any over-estimate
    wallet.save(update_fields=["on_hold", "balance"])

    # Guard earnings credit
    guard_wallet =
        guard.wallet.__class__.objects.select_for_update().get(guard=guard)
    platform_fee = final_amount * Decimal("0.15")
    guard_earnings = final_amount - platform_fee
    guard_wallet.balance += guard_earnings
    guard_wallet.save(update_fields=["balance"])

    return booking
```

3.4 Cancel Booking — Refund Policy

```
@transaction.atomic
def cancel_booking(booking_id: int, cancelled_by, reason: str =
    ""):
```



```

booking = (
    Booking.objects
    .select_for_update()
    .select_related("user__wallet")
    .get(id=booking_id)
)

if booking.status not in ("pending", "accepted"):
    raise ValueError(f"Cannot cancel booking in status: {booking.status}")

now = timezone.now()
hours_until_start = (booking.scheduled_at -
    now).total_seconds() / 3600

# Cancellation policy: full refund if >2h before start
refund_amount = booking.estimated_amount if hours_until_start
    > 2 else Decimal("0")

booking.status = "cancelled"
booking.cancelled_at = now
booking.cancelled_by = cancelled_by
booking.cancel_reason = reason
booking.save(update_fields=["status", "cancelled_at",
    "cancelled_by", "cancel_reason"])

wallet = booking.user.wallet
wallet.on_hold -= booking.estimated_amount
wallet.balance += booking.estimated_amount # release full
hold
wallet.save(update_fields=["on_hold", "balance"])

if refund_amount > 0:
    WalletTransaction.objects.create(
        wallet=wallet,
        booking=booking,
        amount=refund_amount,
        transaction_type="refund",
        description=f"Refund for cancelled booking
        #{booking.id}",
    )

return booking, refund_amount

```

3.5 Get Active Booking for Guard

```

def get_active_booking_for_guard(guard):
    """Single active booking for a guard (business rule: only one
    at a time)."""
    return (
        Booking.objects
        .filter(guard=guard, status="active")
        .select_related("user", "user__profile")
    )

```

```
        .first()
    )
```

3.6 Booking History — Cursor-Based Pagination

```
from bookings.models import Booking
```

```
def get_booking_history(user, cursor_created_at=None, limit: int
                        = 20):
    """
    Cursor-based pagination using `created_at` as the cursor.
    Avoids OFFSET performance degradation on large tables.
    """
    qs = (
        Booking.objects
        .filter(user=user)
        .select_related("guard", "guard__tier")
        .only(
            "id", "status", "scheduled_at", "final_amount",
            "estimated_amount", "created_at",
            "guard__full_name", "guard__profile_photo",
            "guard__tier__name",
        )
        .order_by("-created_at")
    )

    if cursor_created_at:
        qs = qs.filter(created_at__lt=cursor_created_at)

    results = list(qs[:limit + 1])
    has_next = len(results) > limit
    return results[:limit], has_next
```

4. Payment / Wallet Queries

4.1 Debit Wallet — Race Condition Safe

```
from django.db import transaction
from payments.models import Wallet, WalletTransaction
from decimal import Decimal

@transaction.atomic
def debit_wallet(user_id: int, amount: Decimal, description: str,
                booking=None):
    """
    Debit a user's wallet balance. Uses SELECT FOR UPDATE to
    prevent
    """
```

```

race conditions when two requests hit simultaneously.
"""
wallet =
    Wallet.objects.select_for_update().get(user_id=user_id)

if wallet.balance < amount:
    raise ValueError(
        f"Insufficient balance. Available: {wallet.balance},
        Required: {amount}"
    )

wallet.balance -= amount
wallet.save(update_fields=["balance", "updated_at"])

txn = WalletTransaction.objects.create(
    wallet=wallet,
    booking=booking,
    amount=-amount,
    transaction_type="debit",
    description=description,
    balance_after=wallet.balance,
)
return txn

```

4.2 Credit Wallet — Razorpay Webhook Top-Up

```

@transaction.atomic
def credit_wallet_topup(user_id: int, amount: Decimal,
    razorpay_payment_id: str):
    """
    Called by the Razorpay webhook handler after payment.captured
    event.
    Idempotent: checks for existing transaction with the same
    payment_id.
    """
    if WalletTransaction.objects.filter(
        external_reference=razorpay_payment_id
    ).exists():
        return None    # already processed – idempotent

    wallet =
        Wallet.objects.select_for_update().get(user_id=user_id)
    wallet.balance += amount
    wallet.save(update_fields=["balance", "updated_at"])

    txn = WalletTransaction.objects.create(
        wallet=wallet,
        amount=amount,
        transaction_type="topup",
        external_reference=razorpay_payment_id,
        description=f"Wallet top-up via Razorpay
        [{razorpay_payment_id}]",
        balance_after=wallet.balance,
    )

```

```
)
return txn
```

4.3 Transaction History with Pagination

```
def get_transaction_history(user, page: int = 1, page_size: int =
    25):
    from django.core.paginator import Paginator

    qs = (
        WalletTransaction.objects
        .filter(wallet__user=user)
        .select_related("booking")
        .order_by("-created_at")
    )
    paginator = Paginator(qs, page_size)
    return paginator.page(page)
```

4.4 Guard Earnings Summary — By Month

```
from django.db.models import Sum
from django.db.models.functions import TruncMonth
from payments.models import GuardWalletTransaction
```

```
def guard_monthly_earnings(guard_id: int):
    return (
        GuardWalletTransaction.objects
        .filter(wallet__guard_id=guard_id,
            transaction_type="earning")
        .annotate(month=TruncMonth("created_at"))
        .values("month")
        .annotate(total_earnings=Sum("amount"))
        .order_by("-month")
    )
```

-- Raw SQL equivalent

```
SELECT
    DATE_TRUNC('month', gwt.created_at) AS month,
    SUM(gwt.amount) AS total_earnings
FROM payments_guardwallettransaction gwt
JOIN payments_guardwallet gw ON gw.id = gwt.wallet_id
WHERE
    gw.guard_id = 42
    AND gwt.transaction_type = 'earning'
GROUP BY month
ORDER BY month DESC;
```

5. Analytics / Aggregation Queries

5.1 Daily Booking Counts – Last 30 Days

```
from django.db.models import Count
from django.db.models.functions import TruncDay
from django.utils import timezone
from datetime import timedelta
from bookings.models import Booking
```

```
def daily_booking_counts():
    since = timezone.now() - timedelta(days=30)
    return (
        Booking.objects
        .filter(created_at__gte=since)
        .annotate(day=TruncDay("created_at"))
        .values("day")
        .annotate(count=Count("id"))
        .order_by("day")
    )
```

5.2 Revenue by Guard Tier

```
from django.db.models import Sum
from bookings.models import Booking
```

```
def revenue_by_tier():
    return (
        Booking.objects
        .filter(status="completed")
        .values("guard__tier__name")
        .annotate(total_revenue=Sum("final_amount"))
        .order_by("-total_revenue")
    )
```

5.3 Average Booking Duration by City

```
SELECT
    b.city,
    AVG(b.actual_hours)                AS avg_duration_hours,
    COUNT(*)                          AS total_bookings
FROM bookings_booking b
WHERE b.status = 'completed'
    AND b.actual_hours IS NOT NULL
GROUP BY b.city
ORDER BY total_bookings DESC;
```

5.4 SOS Events by Hour of Day (Peak Safety Hours)

```
SELECT
    EXTRACT(HOUR FROM se.triggered_at) AS hour_of_day,
    COUNT(*) AS sos_count
FROM sos_sosevent se
GROUP BY hour_of_day
ORDER BY hour_of_day;

from django.db.models import Count
from django.db.models.functions import ExtractHour
from sos.models import SOSEvent

def sos_by_hour():
    return (
        SOSEvent.objects
        .annotate(hour=ExtractHour("triggered_at"))
        .values("hour")
        .annotate(sos_count=Count("id"))
        .order_by("hour")
    )
```

5.5 Guard Utilization Rate

```
-- Completed bookings per guard / total available guards
WITH guard_stats AS (
    SELECT
        g.id,
        g.full_name,
        COUNT(b.id) FILTER (WHERE b.status = 'completed') AS
        completed_bookings,
        COUNT(b.id) AS
        total_bookings
    FROM guards_guard g
    LEFT JOIN bookings_booking b ON b.guard_id = g.id
    WHERE g.is_active = TRUE
    GROUP BY g.id, g.full_name
)
SELECT
    id,
    full_name,
    completed_bookings,
    total_bookings,
    ROUND(
        completed_bookings::numeric / NULLIF(total_bookings, 0) *
        100,
        2
    ) AS utilization_pct
FROM guard_stats
ORDER BY utilization_pct DESC;
```

5.6 Top 10 Guards by Revenue

```
from django.db.models import Sum
from guards.models import Guard

def top_guards_by_revenue(limit: int = 10):
    return (
        Guard.objects
        .filter(bookings__status="completed")
        .annotate(total_revenue=Sum("bookings__final_amount"))
        .select_related("tier")
        .order_by("-total_revenue")[:limit]
    )
```

6. Review & Rating Queries

6.1 Latest Reviews for a Guard

```
from reviews.models import Review

def get_guard_reviews(guard_id: int, limit: int = 10):
    return (
        Review.objects
        .filter(booking__guard_id=guard_id)
        .select_related("booking__user", "booking__user__profile")
        .only(
            "id", "rating", "comment", "created_at",
            "booking__user__full_name",
            "booking__user__profile__photo",
        )
        .order_by("-created_at")[:limit]
    )
```

6.2 Rating Distribution Histogram

```
from django.db.models import Count

def rating_distribution(guard_id: int):
    """Returns count of each star rating (1-5) for a guard."""
    return (
        Review.objects
        .filter(booking__guard_id=guard_id)
        .values("rating")
        .annotate(count=Count("id"))
    )
```

```

        .order_by("rating")
    )

-- Raw SQL with guaranteed 1-5 rows even if count=0
WITH star_series AS (
    SELECT generate_series(1, 5) AS star
)
SELECT
    s.star,
    COUNT(r.id) AS review_count
FROM star_series s
LEFT JOIN reviews_review r
    ON r.rating = s.star
    AND r.booking_id IN (
        SELECT id FROM bookings_booking WHERE guard_id = 42
    )
GROUP BY s.star
ORDER BY s.star;

```

6.3 Guards Below Rating Threshold (Admin Moderation Queue)

```

from django.db.models import Avg, Count

def guards_below_rating_threshold(threshold: float = 3.0,
    min_reviews: int = 5):
    return (
        Guard.objects
        .filter(is_active=True)
        .annotate(
            avg_rating=Avg("bookings__review__rating"),
            review_count=Count("bookings__review"),
        )
        .filter(avg_rating__lt=threshold,
            review_count__gte=min_reviews)
        .order_by("avg_rating")
    )

```

7. Admin Dashboard Queries

7.1 Platform Metrics Snapshot

```

-- Single query using subqueries – returns one row with all key
  metrics
SELECT
    (SELECT COUNT(*) FROM users_user WHERE is_active =
        TRUE) AS total_active_users,

```



```

(SELECT COUNT(*) FROM guards_guard WHERE is_active =
    TRUE) AS total_active_guards,
(SELECT COUNT(*) FROM bookings_booking
    WHERE created_at::date =
        CURRENT_DATE) AS
    bookings_today,
(SELECT COALESCE(SUM(final_amount), 0) FROM bookings_booking
    WHERE status = 'completed'
    AND completed_at::date =
        CURRENT_DATE) AS revenue_today,
(SELECT COUNT(*) FROM bookings_booking WHERE status =
    'active') AS active_bookings_now,
(SELECT COUNT(*) FROM sos_sosevent WHERE status =
    'active') AS active_sos_now;

from django.db.models import Count, Sum, Q
from django.utils import timezone

def platform_snapshot():
    today = timezone.now().date()
    from users.models import User
    from guards.models import Guard
    from bookings.models import Booking
    from sos.models import SOSEvent

    return {
        "total_active_users":
            User.objects.filter(is_active=True).count(),
        "total_active_guards":
            Guard.objects.filter(is_active=True).count(),
        "bookings_today":
            Booking.objects.filter(created_at__date=today).count(),
        "revenue_today":
            Booking.objects.filter(
                status="completed",
                completed_at__date=today,
            ).aggregate(total=Sum("final_amount"))
        ["total"] or 0,
        "active_bookings_now":
            Booking.objects.filter(status="active").count(),
        "active_sos_now":
            SOSEvent.objects.filter(status="active").count(),
    }

```

7.2 Pending Verification Documents

```

from guards.models import GuardDocument

def pending_verifications():
    return (
        GuardDocument.objects
        .filter(status="pending")
        .select_related("guard", "guard__user")
    )

```

```
        .order_by("submitted_at")
    )
```

7.3 Active SOS Events with Context

```
from sos.models import SOSEvent
```

```
def active_sos_events():
    return (
        SOSEvent.objects
        .filter(status="active")
        .select_related(
            "booking",
            "booking__user",
            "booking__guard",
        )
        .order_by("triggered_at")
    )
```

8. Tracking / Location Queries

8.1 Replay Booking Path

```
from tracking.models import LocationSnapshot
```

```
def booking_path(booking_id: int):
    """Returns all GPS snapshots for a booking ordered
    chronologically."""
    return (
        LocationSnapshot.objects
        .filter(booking_id=booking_id)
        .only("latitude", "longitude", "heading", "speed",
            "recorded_at")
        .order_by("recorded_at")
    )
```

8.2 Last Known Location for Each Active Guard

```
-- Uses DISTINCT ON to get the latest snapshot per guard
SELECT DISTINCT ON (ls.guard_id)
    ls.guard_id,
    ls.latitude,
    ls.longitude,
    ls.recorded_at
FROM tracking_locationsnapshot ls
JOIN bookings_booking b ON b.id = ls.booking_id
```

```

WHERE b.status = 'active'
ORDER BY ls.guard_id, ls.recorded_at DESC;

from tracking.models import LocationSnapshot

def last_known_locations():
    # Uses subquery to get max recorded_at per guard
    from django.db.models import OuterRef, Subquery
    latest_ts = (
        LocationSnapshot.objects
        .filter(guard_id=OuterRef("guard_id"))
        .order_by("-recorded_at")
        .values("recorded_at")[:1]
    )
    return (
        LocationSnapshot.objects
        .filter(recorded_at=Subquery(latest_ts))
        .select_related("guard")
    )

```

8.3 Heatmap Data — ST_SnapToGrid

```

-- Aggregate location points into ~500m grid cells for heatmap
  rendering
SELECT
    ST_AsGeoJSON(ST_SnapToGrid(location::geometry, 0.005)) AS
        cell,
    COUNT(*) AS point_count
FROM tracking_locationsnapshot
WHERE recorded_at >= NOW() - INTERVAL '7 days'
GROUP BY cell
ORDER BY point_count DESC
LIMIT 500;

```

9. Search Queries

9.1 Full-Text Guard Search

```

from django.contrib.postgres.search import SearchQuery,
    SearchVector
from guards.models import Guard

def full_text_guard_search(query_str: str):
    """
    Guards have a `search_vector` tsvector column kept up-to-date
    by a PostgreSQL trigger on full_name, bio, skills.
    """

```

```

query = SearchQuery(query_str, search_type="websearch",
                    config="english")
return (
    Guard.objects
        .filter(search_vector=query, is_active=True)
        .order_by("-average_rating")
)

-- Direct SQL equivalent using the pre-computed tsvector column
SELECT id, full_name, average_rating
FROM guards_guard
WHERE search_vector @@ to_tsquery('english', 'bangalore & armed')
    AND is_active = TRUE
ORDER BY ts_rank(search_vector, to_tsquery('english',
    'bangalore & armed')) DESC;

-- Create/maintain the search_vector column
ALTER TABLE guards_guard ADD COLUMN search_vector tsvector;

UPDATE guards_guard
SET search_vector = to_tsvector('english',
    COALESCE(full_name, '') || ' ' ||
    COALESCE(bio, '') || ' ' ||
    COALESCE(array_to_string(skills, ' '), ''))
);

CREATE INDEX idx_guard_search_vector ON guards_guard USING GIN
    (search_vector);

```

9.2 Phone Number Lookup

```

from users.models import User

def lookup_by_phone(phone: str):
    """Case-insensitive phone number lookup (normalise E.164
    format first)."""
    return
        User.objects.filter(phone_number__iexact=phone.strip()).first()

```

9.3 Guard Filter by Skills (Array Overlap)

```

from guards.models import Guard

def guards_with_skills(required_skills: list[str]):
    """
    Returns guards who have ALL of the required skills.
    skills is a PostgreSQL ArrayField(CharField).
    __overlap = any match (OR)
    __contains = all must be present (AND)
    """

```

```

    return Guard.objects.filter(
        skills__contains=required_skills,    # guard has ALL
        required_skills
        is_active=True,
    )

-- Array overlap (guard has ANY of the skills)
SELECT id, full_name, skills
FROM guards_guard
WHERE skills && ARRAY['armed', 'cctv']::varchar[]
    AND is_active = TRUE;

-- Array contains (guard has ALL of the skills)
SELECT id, full_name, skills
FROM guards_guard
WHERE skills @> ARRAY['armed', 'first_aid']::varchar[]
    AND is_active = TRUE;

```

10. Performance Tips

10.1 Defer Unnecessary Columns

```

# Bad – loads all columns including large text/blob fields
Guard.objects.filter(is_active=True)

# Good – load only what the serializer needs
Guard.objects.filter(is_active=True).only(
    "id", "full_name", "average_rating", "hourly_rate", "location"
)

# Alternative – exclude known heavy columns
Guard.objects.filter(is_active=True).defer("bio",
    "background_check_report")

```

10.2 Avoid N+1 — select_related and prefetch_related

```

# FK / OneToOne → select_related (SQL JOIN)
Booking.objects.select_related("user", "guard", "guard__tier")

# Reverse FK / M2M → prefetch_related (separate query + Python
    join)
Guard.objects.prefetch_related("skills_set", "reviews")

# Custom Prefetch for filtered sub-queryset
from django.db.models import Prefetch
Guard.objects.prefetch_related(
    Prefetch(
        "bookings",

```

```

        queryset=Booking.objects.filter(status="completed").order_by("-
        completed_at"),
        to_attr="recent_completed",
    )
)

```

10.3 Large Querysets – Use iterator()

```

# Bad – loads all 100k rows into memory at once
for booking in Booking.objects.filter(status="completed"):
    process(booking)

# Good – streams in chunks of `chunk_size` rows
for booking in
    Booking.objects.filter(status="completed").iterator(chunk_size=500):
    process(booking)

```

10.4 Debug Queries in Development

```

# settings/local.py – enable query logging
LOGGING = {
    "loggers": {
        "django.db.backends": {
            "level": "DEBUG",
            "handlers": ["console"],
        }
    }
}

# In a shell or test – inspect executed queries
from django.db import connection, reset_queries
reset_queries()

Guard.objects.filter(is_active=True)[:5]

for q in connection.queries:
    print(q["sql"])
    print(f"    Time: {q['time']}s")

```

10.5 Query Profiling Tools

Tool	Setup	Use
django-debug-toolbar	pip install django-debug-toolbar	Browser panel showing all queries per request
django-silk	pip install django-silk	Request/query

Tool	Setup	Use
		profiling with persistent store
EXPLAIN ANALYZE	connection.cursor().execute("EXPLAIN ANALYZE ...")	Raw PostgreSQL query plan
pg_stat_statements	PostgreSQL extension	Aggregate slow query stats across all requests

Quick EXPLAIN ANALYZE from Django shell

```
from django.db import connection
```

```
with connection.cursor() as cursor:
```

```
    cursor.execute("""
```

```
        EXPLAIN (ANALYZE, BUFFERS, FORMAT TEXT)
```

```
        SELECT id FROM guards_guard
```

```
        WHERE ST_DWithin(location::geography, ST_MakePoint(77.59,
```

```
        12.97)::geography, 5000)
```

```
    """)
```

```
    for row in cursor.fetchall():
```

```
        print(row[0])
```

10.6 Index Checklist

-- Verify all critical indexes exist

```
SELECT
```

```
    schemaname,
```

```
    tablename,
```

```
    indexname,
```

```
    indexdef
```

```
FROM pg_indexes
```

```
WHERE tablename IN (
```

```
    'guards_guard',
```

```
    'bookings_booking',
```

```
    'tracking_locationsnapshot',
```

```
    'payments_wallettransaction'
```

```
)
```

```
ORDER BY tablename, indexname;
```

-- Must-have indexes for b-secure

```
CREATE INDEX CONCURRENTLY idx_booking_guard_status
```

```
ON bookings_booking (guard_id, status);
```

```
CREATE INDEX CONCURRENTLY idx_booking_user_created
```

```
ON bookings_booking (user_id, created_at DESC);
```

```
CREATE INDEX CONCURRENTLY idx_location_snapshot_booking_time
  ON tracking_locationsnapshot (booking_id, recorded_at DESC);

CREATE INDEX CONCURRENTLY idx_guard_location_gist
  ON guards_guard USING GIST (location);

CREATE INDEX CONCURRENTLY idx_guard_active_available
  ON guards_guard (is_active, is_available, is_verified)
  WHERE is_active = TRUE;

CREATE INDEX CONCURRENTLY idx_wallet_txn_wallet_created
  ON payments_wallettransaction (wallet_id, created_at DESC);
```